



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 1

Especificación de TADs

November 16, 2025

Algoritmos y Estructuras de Datos

JavaNation

Integrante	LU	Correo electrónico
Chioli, Lautaro	32/25	lautaro.chioli@gmail.com
Temelini, Mateo	1311/24	mateotemelini@gmail.com



Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (+54 +11) 4576-3300

<http://www.exactas.uba.ar>

Especificación del TAD

posición ES tuple $\langle \mathbb{Z}, \mathbb{Z} \rangle$

ejercicio ES struct $\langle ej: \mathbb{Z}, rta: \mathbb{Z} \rangle$

examen ES conj $\langle ejercicio \rangle$

alumno ES struct $\langle pos: posición, examen: examen, entregó: bool \rangle$

TAD EdR {

obs solución : *examen*

obs estudiantes : conj $\langle alumno \rangle$

```
proc EdR(in columnas :  $\mathbb{Z}$ , in resolución : examen, in alumnado : conj $\langle alumno \rangle$ ) : EdR
{
  requiere {  $|alumnado| > 0 \wedge respuestasVálidas(resolución) \wedge$ 
    alumnadoCorrecto(alumnado, columnas) }
  asegura { res.solución = resolución  $\wedge$  res.estudiantes = alumnado }
}
```

```
pred respuestasVálidas(resolución : examen) {
  ( $\forall t : ejercicio$ ) ( $t \in resolución \rightarrow_L 0 \leq t.rta < 10$ )
}
pred alumnadoCorrecto(alumnado : conj $\langle alumno \rangle$ , columnas :  $\mathbb{Z}$ ) {
  alumnosEnRango(alumnado, columnas)  $\wedge$  alumnosSeparados(alumnado)  $\wedge$ 
  alumnosPresentes(alumnado)  $\wedge$  alumnosEnBlanco(alumnado)
}
pred alumnosEnRango(est : conj $\langle alumno \rangle$ , columnas :  $\mathbb{Z}$ ) {
  ( $\forall a : alumno$ ) ( $a \in est \rightarrow_L (0 \leq a.pos_0 < columnas \wedge 0 \leq a.pos_1 < columnas)$ )
}
pred alumnosSeparados(est : conj $\langle alumno \rangle$ ) {
  ( $\forall a, b : alumno$ ) ( $((a \in est \wedge b \in est \wedge a.pos_0 = b.pos_0 \wedge a \neq b) \rightarrow_L$ 
     $|a.pos_1 - b.pos_1| \geq 2)$ )
}
pred alumnosPresentes(est : conj $\langle alumno \rangle$ ) {
  ( $\forall a : alumno$ ) ( $a \in est \rightarrow_L a.entregó = false$ )
}
pred alumnosEnBlanco(est : conj $\langle alumno \rangle$ ) {
  ( $\forall a : alumno$ ) ( $a \in est \rightarrow_L a.examen = \langle \rangle$ )
}
```

```
proc igualdad(in edr1 : EdR, in edr2 : EdR) : bool {
  requiere { True }
  asegura { res = true  $\leftrightarrow$  mismaSolución(edr1.solución, edr2.solución)  $\wedge$ 
    mismosalumnos(edr1.estudiantes, edr2.estudiantes) }
}
```

```

pred mismaSolución( $s_1 : examen$ ,  $s_2 : examen$ ) {
  ( $\forall t : ejercicio$ ) ( $t \in s_1 \leftrightarrow t \in s_2$ )
}

pred mismosalumnos( $est_1 : conj\langle alumno \rangle$ ,  $est_2 : conj\langle alumno \rangle$ ) {
  ( $\forall a : alumno$ ) ( $a \in est_1 \leftrightarrow a \in est_2$ )
}

proc copiarse(inout  $edr : EdR$ , in  $a : alumno$ , in  $b : alumno$ ) {
  requiere {  $ambosPresentes(a, b, edr.estudiantes) \wedge posibleCopia(edr, a, b) \wedge edr = edr_0$  }
  asegura {  $modificarExámenes(edr_0.estudiantes, edr.estudiantes, a, b) \wedge edr.solución = edr_0.solución$  }
}

pred ambosPresentes( $a : alumno$ ,  $b : alumno$ ,  $est : conj\langle alumno \rangle$ ) {
   $a \in est \wedge b \in est \wedge a.entregó = false \wedge b.entregó = false$ 
}

pred posibleCopia( $edr : EdR$ ,  $a : alumno$ ,  $b : alumno$ ) {
   $|a.examen| < |edr.solución| \wedge ejercicioDisponible(a.examen, b.examen) \wedge posicionesCercanas(a.pos, b.pos)$ 
}

pred posicionesCercanas( $p : posición$ ,  $q : posición$ ) {
   $(p_0 = q_0 \wedge |p_1 - q_1| \leq 2) \vee (p_0 = q_0 - 1 \wedge |p_1 - q_1| \leq 1)$ 
}

pred ejercicioDisponible( $s_1 : examen$ ,  $s_2 : examen$ ) {
  ( $\exists punto : ejercicio$ ) ( $punto \in s_2 \wedge \neg(\exists t \in s_1) (t.ej = punto.ej)$ )
}

pred modificarExámenes( $est_0 : conj\langle alumno \rangle$ ,  $est : conj\langle alumno \rangle$ ,  $a : alumno$ ,  $b : alumno$ ) {
  ( $\exists nuevo \in b.examen$ ) ( $\neg(\exists t \in a.examen) (t.ej = nuevo.ej) \wedge_L exámenesActualizados(est_0, est, a, nuevo)$ )
}

pred exámenesActualizados( $est_0 : conj\langle alumno \rangle$ ,  $est : conj\langle alumno \rangle$ ,  $a : alumno$ ,  $nuevo : ejercicio$ ) {
   $|est_0| = |est| \wedge_L (\forall b \in est_0) ((b \neq a \rightarrow_L b \in est) \wedge (b = a \rightarrow_L (b \notin est \wedge_L (\exists c \in est) (c.pos = a.pos \wedge c.examen = a.examen \cup \{nuevo\} \wedge c.entregó = false \wedge |c.examen| = |a.examen| + 1))))$ 
}

proc consultarDarkWeb(inout  $edr : EdR$ , in  $dark : examen$ , in  $entradas : \mathbb{Z}$ ) {
  requiere {  $examenCompatible(dark, edr.solución) \wedge entradas > 0 \wedge edr = edr_0$  }
  asegura {  $cantidadExamenDark(edr.estudiantes, dark) \leq entradas \wedge estudiantesConservados(edr_0.estudiantes, edr.estudiantes, dark) \wedge edr.solución = edr_0.solución$  }
}

```

```

pred examenCompatible(dark : examen, sol : examen) {
  |dark| = |sol|  $\wedge_L$  ( $\forall t : \text{ejercicio}$ ) ( $t \in \text{dark} \rightarrow_L (\exists k \in \text{sol}) (t.ej = k.ej \wedge 0 \leq t.rta < 10)$ )
}
pred estudiantesConservados(est0 : conj<alumno>, est : conj<alumno>, dark : examen)
{
  |est| = |est0|  $\wedge_L$  ( $\forall a : \text{alumno}$ ) ( $a \in \text{est} \rightarrow_L$ 
  (mismaSolución(a.examen, dark)  $\wedge$  ( $\exists b \in \text{est}_0$ ) ( $b.pos = a.pos \wedge b.entregó = a.entregó$ ))  $\vee$ 
  ( $a \in \text{est}_0 \wedge \neg \text{mismaSolución}(a.examen, dark)$ ))
}
aux cantidadExamenDark(est : conj<alumno>, dark : examen) :  $\mathbb{Z} =$ 
 $\sum_{a \in \text{est}} (\text{IfThenElse}(\text{mismaSolución}(a.examen, dark), 1, 0))$ 

```

```

proc resolver(inout edr : EdR, in a : alumno, in pasos : seq<examen>) : seq<examen> {
  requiere {  $a \in \text{edr.estudiantes} \wedge a.entregó = \text{false} \wedge a.examen = \langle \rangle \wedge$ 
  secuenciaCorrecta(pasos, edr.solución)  $\wedge \text{edr} = \text{edr}_0$  }
  asegura { edr.solución = edr0.solución  $\wedge \text{resVálido}(\text{pasos}, \text{edr}_0.\text{solución}, \text{res}) \wedge$ 
  actualizarExámenes(edr0.estudiantes, edr.estudiantes, a, res) }
}

```

```

pred secuenciaCorrecta(pasos : seq<examen>, sol : examen) {
  ( $1 \leq |\text{pasos}| \leq |\text{sol}| \wedge_L \text{pasos}[0] = \langle \rangle$ )  $\wedge_L$ 
  ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |\text{pasos}| - 1 \rightarrow_L \text{ejercicioAgregado}(\text{pasos}[i], \text{pasos}[i + 1]), \text{sol}) \wedge$ 
  ( $\forall i : \mathbb{Z}$ ) ( $0 \leq i < |\text{pasos}| \rightarrow_L \text{respuestasEnRango}(\text{pasos}[i], \text{sol})$ )
}
pred ejercicioAgregado(paso : examen, siguiente : examen, sol : examen) {
  |siguiente| = |paso| + 1  $\wedge_L$  ( $\forall t : \text{ejercicio}$ ) ( $t \in \text{paso} \rightarrow_L t \in \text{siguiente}$ )  $\wedge$ 
  ( $\exists j : \text{ejercicio}$ ) (ejercicioVálido(paso, sol, j)  $\wedge j \in \text{siguiente}$ )
}
pred respuestasEnRango(paso : examen, sol : examen) {
  ( $\forall t : \text{ejercicio}$ ) ( $t \in \text{paso} \rightarrow_L (\exists j \in \text{sol}) (j.ej = t.ej \wedge 0 \leq t.rta < 10)$ )
}

```

```

pred resVálido(pasos : seq<examen>, sol : examen, res : seq<examen>) {
  |res| = |pasos| + 1  $\wedge_L$  ( $(\forall i : \mathbb{Z}) (0 \leq i < |\text{pasos}| \rightarrow_L \text{res}[i] = \text{pasos}[i]) \wedge$ 
  ( $\exists \text{adicional} : \text{ejercicio}$ ) (ejercicioVálido(pasos[|pasos| - 1], sol, adicional)  $\wedge_L$ 
  res[|res| - 1] = pasos[|pasos| - 1] + + < adicional >))
}
pred ejercicioVálido(paso : examen, sol : examen, adicional : ejercicio) {
  ( $\forall t : \text{ejercicio}$ ) ( $t \in \text{paso} \rightarrow_L t.ej \neq \text{adicional.ej}$ )  $\wedge$ 
  ( $\exists j \in \text{sol}$ ) ( $j.ej = \text{adicional.ej} \wedge 0 \leq \text{adicional.rta} < 10$ )
}

```

```

pred actualizarExámenes( $est_0 : \text{conj}\langle \text{estudiantes} \rangle$ ,  $est : \text{conj}\langle \text{estudiantes} \rangle$ ,  $a : \text{alumno}$ ,  $res : \text{seq}\langle \text{examen} \rangle$ ) {
   $|est_0| = |est| \wedge_L (\forall b \in est_0) ((b \neq a \rightarrow_L b \in est) \wedge$ 
     $(b = a \rightarrow_L (b \notin est \wedge_L (\exists c \in est) (c.pos = a.pos \wedge c.examen = res[|res| - 1] \wedge$ 
       $c.entregó = false))))$ 
}

proc entregar(inout  $edr : \text{EdR}$ , in  $a : \text{alumno}$ ) {
  requiere {  $a \in edr.estudiantes \wedge a.entregó = false \wedge edr = edr_0$  }
  asegura {  $\text{estudianteQuitado}(edr_0.estudiantes, a, edr.estudiantes) \wedge edr.solución =$ 
     $edr_0.solución$  }
}

pred estudianteQuitado( $est_0 : \text{conj}\langle \text{alumno} \rangle$ ,  $a : \text{alumno}$ ,  $est : \text{conj}\langle \text{estudiantes} \rangle$ ) {
   $|est_0| = |est| \wedge_L (\forall b \in est_0) ((b \neq a \rightarrow_L b \in est) \wedge$ 
     $(b = a \rightarrow_L (b \notin est \wedge_L (\exists c \in est) (c.pos = a.pos \wedge c.examen = a.examen \wedge$ 
       $c.entregó = true))))$ 
}

proc chequearCopias(in  $edr : \text{EdR}$ ) :  $\text{seq}\langle \text{alumno} \rangle$  {
  requiere {  $\text{aulaVacía}(edr.estudiantes)$  }
  asegura {  $(\forall a \in edr.estudiantes) (a \in res \leftrightarrow \text{sospechoso}(a, edr.estudiantes, edr.solución))$  }
}

pred aulaVacía( $est : \text{conj}\langle \text{estudiantes} \rangle$ ) {
   $(\forall a : \text{alumno}) (a \in est \rightarrow_L a.entregó = true)$ 
}

pred sospechoso( $a : \text{alumno}$ ,  $est : \text{conj}\langle \text{alumno} \rangle$ ,  $sol : \text{examen}$ ) {
   $\text{cantidadExámenesIguales}(a, est) > (|est| - 1) / 4 \vee \text{examenCopiado}(a, est, sol)$ 
}

pred examenCopiado( $a : \text{alumno}$ ,  $est : \text{conj}\langle \text{alumno} \rangle$ ,  $sol : \text{examen}$ ) {
   $(\exists b \in est) (a \neq b \wedge \text{posicionesCercanas}(a.pos, b.pos) \wedge$ 
     $\text{cantidadRespuestasIguales}(a.examen, b.examen) > 0.6 * |sol|)$ 
}

aux cantidadExámenesIguales( $a : \text{alumno}$ ,  $est : \text{conj}\langle \text{alumno} \rangle$ ) :  $\mathbb{Z} =$ 
   $\sum_{b \in est} (\text{IfThenElse}(a \neq b \wedge \text{mismaSolución}(a.examen, b.examen), 1, 0))$ 

aux cantidadRespuestasIguales( $s_1 : \text{examen}$ ,  $s_2 : \text{examen}$ ) :  $\mathbb{Z} =$ 
   $\sum_{t \in s_1} (\text{IfThenElse}(\text{ejercicioCoincidente}(t, s_2), 1, 0))$ 

pred ejercicioCoincidente( $t : \text{ejercicio}$ ,  $s_2 : \text{examen}$ ) {
   $(\exists k : \text{ejercicio}) (k \in s_2 \wedge t = k)$ 
}

```

```

proc corregir(in edr : EdR) : seq⟨struct < a : alumno, n : ℝ >⟩ {
  requiere { aulaVacía(edr.estudiantes) }
  asegura { correcciones(res, edr.estudiantes, edr.solución) }
}
pred correcciones(res : seq⟨struct < a : alumno, n : ℝ >⟩, est : conj⟨alumno⟩, sol :
examen) {
  (∀ a ∈ est)
  ((sospechoso(a, est, sol) →L ¬notaAsignada(a, sol, res)) ∧
  (¬sospechoso(a, est, sol) →L notaAsignada(a, sol, res)))
}
pred notaAsignada(a : alumno, sol : examen, res : seq⟨struct < a : alumno, n : ℝ >⟩)
{
  (∃ k : ℤ) (0 ≤ k < |res| ∧L (res[k].a = a ∧
  res[k].n = cantidadRespuestasIguales(a.examen, sol) / |sol|))
}

```